Object-Oriented Harness: Evolving Agent Harnesses through Object-Oriented Design

https://arxiv.org/abs/2610.28380

1. Problem

- 해결하려는 문제: 단일 Agent Harness로 다양한 유형의 복잡한 작업을 최적화하기 어려운 문제 해결 목적임.
- 중요성: 특정 작업을 개선하는 변경 사항이 다른 작업의 성능을 저하시킬 수 있어 작업별 최적화 필요함.

2. Method

- 핵심 아이디어: 객체지향 설계를 활용해 작업별 특화 Harness를 생성하고, 적합한 Harness를 선택하는 방식임.
- 주요 구성 요소
  - Harness Evolution: 실행 기록을 분석해 유사 작업을 분류하고 특화 Harness 생성함.
  - Inheritance: 기존 Harness의 기능을 상속해 검증된 동작을 재사용하고 필요한 부분만 수정함.
  - Harness Router: 작업 특성에 맞는 Harness를 선택함.
- 기존 방법과의 차이: 단일 Harness를 개선하는 대신, 상속 기반 계층 구조와 작업별 라우팅을 결합함.
- 실행 과정: 기본 실행 → 작업 클러스터링 → 특화 Harness 생성 및 평가 → 적합한 Harness 선택 순서임.

3. Experiments & Results

- 실험 환경: AutomationBench, Workspace-Bench, Toolathlon-GYM 등에서 평가함.
- 주요 결과

실험	성능
AutomationBench 기본 모델	61.62%
AutomationBench OO-Harness	77.41%
Adaptive Auto-Harness	75.40%
Workspace-Bench Base-reset	29.39%
Workspace-Bench Flat 복사 방식	35.60%
Workspace-Bench OO 상속 방식	40.56%

- AutomationBench에서 기본 모델 대비 15.79%p 향상함.
- Adaptive Auto-Harness 대비 2.01%p 향상함.
- Workspace-Bench에서 상속 방식이 단순 복사 방식보다 4.96%p 높은 성능을 기록함.
- 작업별 Harness 특화와 상속 구조가 성능 개선에 효과적임을 보임.

4. Insight

- 핵심 기여: 객체지향 상속과 작업별 라우팅을 활용해 Agent Harness를 작업에 맞게 진화시키는 방법 제안함.
- 한계점
  - 특화 Harness 생성 및 선택에 따른 추가 추론 비용 발생 가능함.
  - 일부 절제 실험만으로 상속 방식의 효과를 일반화하기 어려움.
  - 새로운 작업 환경에서의 일반화 성능에 대한 추가 검증 필요함.
- 가져갈 포인트
  - Agent 성능은 모델뿐 아니라 프롬프트, 도구 사용 방식, 실행 흐름을 설계하는 Harness에도 영향을 받음.
  - 실패 기록을 활용해 특정 작업의 성능을 개선하고, 검증된 기능은 상속해 재사용하는 설계가 유용함.
  - 실제 적용 시 성공률뿐 아니라 토큰 사용량, 실행 시간, 작업별 성능도 함께 평가해야 함.